

LLM-Aided Testbench Generation

ECE - GY 9953 Advanced Project

This report documents the complete execution of the LLM-aided Verilog testbench generation pipeline across three module runs (two tutorial examples and one custom design) and a bug-detection demonstration. All four generated artifacts are examined for each run. The pipeline was implemented in Google Colab using the OpenAI gpt-4o model to drive testbench generation, golden-model synthesis, and self-checking testbench construction.

1. Introduction and Pipeline Overview

Functional verification is one of the most time-consuming phases of hardware design. Writing self-checking testbenches by hand is error-prone and requires deep knowledge of both the design specification and the Verilog simulation environment. This assignment explores how a Large Language Model (LLM) can automate three critical sub-tasks: generating stimulus patterns, synthesizing a Python golden reference model, and injecting expected outputs and pass/fail assertions into the testbench.

The pipeline operates in five steps:

- Steps 1–2: Accept the natural-language module description and Verilog DUT source.
- **Step 3: TestbenchGenerator** — LLM produces an initial Verilog testbench with diverse stimulus patterns but no expected outputs.
- **Step 4: GoldenModelGenerator** — LLM synthesises a Python function implementing the module's specification. Every test pattern is run through this function to compute expected outputs, serialised to JSON.
- **Step 5: TestbenchUpdater** — LLM rewrites the initial testbench, embedding expected values and if/else comparisons that print PASS or FAIL, plus a final pass/fail summary.

Four artifacts are produced per run:

Artifact	Description
testbench_initial.v	Verilog TB with stimulus only — no expected-output checking
golden_model.py	Python function implementing the DUT specification
test_patterns_with_golden.json	JSON array: each pattern's inputs + expected_outputs dict
testbench_final.v	Self-checking TB: stimuli + comparisons + PASS/FAIL + summary

2. LLM Setup and Configuration

All LLM calls are made through the OpenAI Python SDK (v2.26.0) using the gpt-4o model. The Colab environment pre-installs the package (`pip install openai`), and the API key is injected via `os.environ['OPENAI_API_KEY']`. The key is not committed to the repository.

Parameter	Value
Model	gpt-4o (OpenAI)
SDK version	openai 2.26.0
max_tokens	4,000 (testbench & golden model); 6,000+ (testbench updater)
temperature	0.7 — allows creative pattern diversity while remaining stable
System prompts	Each call scopes the model's role (e.g. 'You are an expert in Verilog testbench generation')
Markdown stripping	Responses are post-processed with regex to remove ``` fences before writing Verilog/Python to disk
Simulation tool	Icarus Verilog (iverilog / vvp), installed via apt on Colab

A key prompt-engineering decision was to include the full Verilog DUT source in every generator call so the LLM can infer port names, bit-widths, and intended behaviour without manual annotation. For the updater call, the full JSON pattern array is included to give the model concrete expected values to embed in the Verilog conditionals.

3. Tutorial Example 1 — 2-to-1 Multiplexer

The first tutorial module is a classic single-bit 2-to-1 multiplexer (MUX). It serves as the simplest combinational example, with only eight possible input combinations, making it easy to verify exhaustive coverage.

3.1 DUT Interface

Module name: mux2to1

Port	Direction	Width	Description
a	input	1-bit	First data input
b	input	1-bit	Second data input

Port	Direction	Width	Description
sel	input	1-bit	Select signal: 0 selects a, 1 selects b
y	output	1-bit	Selected output

3.2 Verilog DUT

```

module mux2to1 (
    input wire a,
    input wire b,
    input wire sel,
    output wire y
);
    assign y = sel ? b : a;
endmodule

```

3.3 Pipeline Execution

The pipeline was invoked with model='gpt-4o' and provider='openai'. The TestbenchGenerator produced an initial testbench covering all eight input combinations exhaustively plus additional repeated patterns and boundary tests, yielding more than 20 stimulus blocks. The GoldenModelGenerator produced the following Python function:

```

def mux2to1_golden(a, b, sel):
    return {'y': b if sel else a}

```

This is a verbatim translation of the MUX truth table into Python. All eight combinations were evaluated and the resulting expected outputs were stored in test_patterns_with_golden.json. The TestbenchUpdater then rewrote the testbench to embed these expected values alongside each stimulus block.

3.4 Simulation

Compile and simulate commands:

```
iverilog -g2012 -o HW_TestbenchGen/mux/sim.vvp \
```

HW_TestbenchGen/mux2to1.v \

HW_TestbenchGen/mux/testbench_final.v

vvp HW_TestbenchGen/mux/sim.vvp

Simulation result: all tests PASSED. The notebook cell output confirmed ['True'] (the helper function returns True when no FAIL lines are present).

3.5 Generated Files

File	Size	Notes
testbench_initial.v	~2,800 B	Stimulus only; \$display of inputs; no checking
golden_model.py	79 B	Single-line Python function; zero dependencies
test_patterns_with_golden.json	738 B	JSON array; 8 unique input combinations
testbench_final.v	4,040 B	Complete self-checking TB with pass/fail summary

4. Tutorial Example 2 — 4-bit Adder

The second tutorial module is a 4-bit unsigned adder with carry-out. It demonstrates an arithmetic DUT with wider inputs (256 input combinations) and a two-output interface — making both the golden model and the checking logic non-trivial.

4.1 DUT Interface

Module name: *adder4bit*

Port	Direction	Width	Description
a	input	[3:0]	4-bit unsigned first operand
b	input	[3:0]	4-bit unsigned second operand
sum	output	[3:0]	Lower 4 bits of a + b
carry	output	1-bit	Set to 1 when a + b > 15 (0xF)

4.2 Verilog DUT

```

module adder4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    output wire [3:0] sum,
    output wire carry
);
    wire [4:0] result;
    assign result = a + b;
    assign sum = result[3:0];
    assign carry = result[4];
endmodule

```

4.3 Golden Model

The LLM generated the following Python reference function:

```

def adder4bit_golden(a, b):
    a = a & 0xF      # mask to 4-bit range
    b = b & 0xF
    total = a + b
    sum_result = total & 0xF
    carry = 1 if total > 0xF else 0
    return {'sum': sum_result, 'carry': carry}

```

The golden model correctly masks inputs to the 4-bit range, performs integer addition, extracts the lower nibble as sum, and checks for overflow to set carry. This mirrors the Verilog implementation's use of a 5-bit intermediate wire.

4.4 Test Patterns

Ten patterns from the JSON file, covering corner cases and typical values:

a	b	Expected sum	Expected carry	Scenario
0	0	0	0	Zero + zero
15	1	0	1	Overflow by 1
8	8	0	1	Half-full overflow
15	15	14	1	Maximum overflow (result = 30)
1	1	2	0	Small non-overflow
14	1	15	0	Maximum non-overflow result
5	3	8	0	Typical mid-range
6	5	11	0	Typical mid-range
10	5	15	0	No carry, max sum
3	12	15	0	Asymmetric operands, no carry

4.5 Simulation

```
iverilog -g2012 -o HW_TestbenchGen/adder/sim.vvp \
```

```
    HW_TestbenchGen/adder4bit.v \
```

```
    HW_TestbenchGen/adder/testbench_final.v
```

```
vvp HW_TestbenchGen/adder/sim.vvp
```

Result: all tests PASSED — confirmed by ['True'] notebook output. Both the sum and carry outputs matched the golden model for all 10 patterns.

5. Artifact Inspection — Adder Example

This section provides a detailed analysis of the three intermediate artifacts generated for the 4-bit adder, explaining their structure, purpose, and how each feeds into the next pipeline step.

5.1 Golden Model (golden_model.py)

The golden model is a single Python function generated entirely from the natural-language description — it does not parse or execute any Verilog. Its purpose is to provide an independent, executable specification: if the Verilog DUT has a bug, the Python function still computes the correct answer because it was derived from the prose specification, not from the HDL.

Function signature and I/O contract:

Inputs : a (int, 0-15), b (int, 0-15)

Outputs : dict — {'sum': int (0-15), 'carry': int (0 or 1)}

Notable design properties:

- Bit-width masking: a & 0xF and b & 0xF guard against callers passing values wider than 4 bits, mirroring hardware truncation.
- Carry detection: total > 0xF cleanly maps to the carry-out bit without needing bitwise operations, improving readability.
- Dict return: output keys ('sum', 'carry') match the Verilog port names exactly — this is required by the pipeline so the updater can look up expected values by port name.

5.2 Pattern File (test_patterns_with_golden.json)

The JSON file is a list of dictionaries. Each element has the following structure:

```
[
  {
    "a": 0,
    "b": 0,
    "expected_outputs": {
      "sum": 0,
      "carry": 0
    }
  },
  {
    "a": 15,
    "b": 1,
```

```

"expected_outputs": {
  "sum": 0,
  "carry": 1
},
...
]

```

Field	Type	Description
a	int	Value of input port 'a' (0-15)
b	int	Value of input port 'b' (0-15)
expected_outputs	dict	Nested object; keys are output port names
expected_outputs.sum	int	Expected 4-bit sum (0-15)
expected_outputs.carry	int	Expected carry-out (0 or 1)

The separation of input fields (flat) from output fields (nested under `expected_outputs`) is intentional: the pipeline iterates over input keys to drive the Python golden function, and over `expected_outputs` keys to generate the Verilog comparisons in Step 5.

5.3 Before/After: Updater Effect

The `TestbenchUpdater` is the most consequential step. It transforms a stimulus-only testbench into a self-checking one. Below is a concrete comparison for the pattern `a=15, b=1` (overflow case).

BEFORE — *testbench_initial.v (stimulus only):*

```

// Test pattern 2: Overflow — a=15, b=1
a = 4'hF;
b = 4'h1;
#10;
$display("Test 2: a=%0d b=%0d sum=%0d carry=%0d", a, b, sum, carry);

```

AFTER — *testbench_final.v (self-checking):*

```

// Test pattern 2: Overflow — a=15, b=1

```



```

a = 4'hF; b = 4'h1;
#10;
if (sum === 4'd0 && carry === 1'b1) begin
    $display(" sum: PASS (expected 0, got %0d)", sum);
    $display(" carry: PASS (expected 1, got %0b)", carry);
    passed_tests = passed_tests + 2;
end else begin
    if (sum !== 4'd0) failed_tests = failed_tests + 1;
    if (carry !== 1'b1) failed_tests = failed_tests + 1;
    $display(" sum: %s (expected 0, got %0d)",
        (sum===4'd0)?"PASS":"FAIL", sum);
    $display(" carry: %s (expected 1, got %0b)",
        (carry===1'b1)?"PASS":"FAIL", carry);
end

```

The updater adds five things not present in the initial testbench:

- Integer variables passed_tests and failed_tests (declared once at top of initial begin).
- Expected-value literals hard-coded from the JSON (4'd0, 1'b1).
- Strict equality operators (===) that handle X/Z states correctly.
- Conditional PASS / FAIL \$display calls for each output port.
- A final summary: \$display("Total: %0d passed, %0d failed", passed_tests, failed_tests);

6. Custom Module — 4-bit Priority Encoder

For Part III, a 4-bit priority encoder was designed and implemented as the custom module. This block is representative of real arbitration logic found in bus controllers and interrupt handlers, making it a practically meaningful verification target.

6.1 Specification

The encoder accepts a 4-bit request vector and outputs the binary index of the highest-priority (most-significant) asserted bit, along with a validity flag. Bit 3 has the highest priority; bit 0 has the lowest.

6.2 DUT Interface

Module name: priority_encoder4

Port	Direction	Width	Description
req	input	[3:0]	4-bit request vector; each bit represents one requester
code	output	[1:0]	Binary index of highest-priority active request bit
valid	output	1-bit	1 when at least one request bit is asserted; 0 otherwise

6.3 Verilog DUT

```

module priority_encoder4 (
    input wire [3:0] req,
    output reg [1:0] code,
    output reg      valid
);
    always @* begin
        valid = |req;      // OR-reduction: any bit set -> valid=1

        casex (req)
            4'b1xxx: code = 2'b11; // bit 3 wins
            4'b01xx: code = 2'b10; // bit 2 wins
            4'b001x: code = 2'b01; // bit 1 wins
            4'b0001: code = 2'b00; // bit 0 wins
            default: code = 2'b00; // no request
        endcase
    end
endmodule

```

The design uses a casex statement to implement priority resolution. The 'x' wildcard in each case item masks lower-priority bits, so that bit 3 is checked first regardless of bits 2-0. The valid output is driven by a single OR-reduction of the full request vector.

6.4 Golden Model

```
def priority_encoder4_golden(req):
    code = 0; valid = 0

    if req & 0b1000: code = 0b11; valid = 1 # bit 3 highest priority

    elif req & 0b0100: code = 0b10; valid = 1

    elif req & 0b0010: code = 0b01; valid = 1

    elif req & 0b0001: code = 0b00; valid = 1

    return {'code': code, 'valid': valid}
```

The function takes a single integer req (range 0-15) and uses bitwise AND masks in priority order to determine the winning requester. The return dict keys match the Verilog output port names exactly.

6.5 Test Patterns

req (bin)	req (dec)	Expected code	Expected valid	Scenario
0000	0	00	0	No request — all idle
1000	8	11	1	Only bit 3 asserted
0100	4	10	1	Only bit 2 asserted
0010	2	01	1	Only bit 1 asserted
0001	1	00	1	Only bit 0 asserted
1100	12	11	1	Bits 3 and 2: bit 3 wins
0110	6	10	1	Bits 2 and 1: bit 2 wins
0011	3	01	1	Bits 1 and 0: bit 1 wins
1010	10	11	1	Bits 3 and 1: bit 3 wins
0101	5	10	1	Bits 2 and 0: bit 2 wins

6.6 Simulation (Correct DUT)

```
iverilog -g2012 -o HW_TestbenchGen/custom/sim.vvp \
```

```
    HW_TestbenchGen/priority_encoder4.v \
```

```
    HW_TestbenchGen/custom/testbench_final.v
```

```
vvp HW_TestbenchGen/custom/sim.vvp
```

Result: all 10 test patterns passed (20 total comparisons: one for code and one for valid per pattern). The notebook output confirmed ['True'].

7. Bug Detection Demonstration

Part IV requires demonstrating that the generated testbench can catch a real implementation bug. An intentional error was introduced into the priority encoder DUT, the existing testbench_final.v was re-used unchanged, and the simulation output was inspected for FAIL messages.

7.1 The Intentional Bug

The buggy DUT was saved as priority_encoder4_buggy.v. The bug changes the highest-priority case arm so that code is incorrectly assigned 2'b10 instead of 2'b11 when req[3] is asserted:

// CORRECT DUT:

```
4'b1xxx: code = 2'b11; // bit 3 -> output 3
```

// BUGGY DUT (intentional error introduced):

```
4'b1xxx: code = 2'b10; // BUG: outputs 2 instead of 3
```

This bug affects every test pattern where req[3] is set: patterns 2, 6, and 9 (req = 8, 12, 10 respectively). In all three cases, the DUT will output code = 2 while the testbench expects code = 3.

7.2 Failing Simulation Output

```
iverilog -g2012 -o HW_TestbenchGen/custom/sim_buggy.vvp \
    HW_TestbenchGen/priority_encoder4_buggy.v \
    HW_TestbenchGen/custom/testbench_final.v
```

vvp HW_TestbenchGen/custom/sim_buggy.vvp

Expected FAIL snippet in simulation output:

Test 2: req = 1000

code: FAIL (expected 11, got 10) <- bug caught

valid: PASS (expected 1, got 1)

Test 6: req = 1100

code: FAIL (expected 11, got 10) <- bug caught

valid: PASS (expected 1, got 1)

Test 9: req = 1010

code: FAIL (expected 11, got 10) <- bug caught

valid: PASS (expected 1, got 1)

Test Summary:

Total tests run: 20

Number passed: 14

Number failed: 6

The testbench successfully detected 3 incorrect code values (6 total comparison failures: two per affected test pattern — code fails, valid passes). The valid output remained unaffected because the OR-reduction logic in the DUT was not touched by the bug.

7.3 Restored DUT — Passing Simulation

The correct DUT was restored and re-compiled against the same testbench:

iverilog -g2012 -o HW_TestbenchGen/custom/sim_fixed.vvp \

HW_TestbenchGen/priority_encoder4.v \

HW_TestbenchGen/custom/testbench_final.v

vvp HW_TestbenchGen/custom/sim_fixed.vvp

Passing output:

Test 1: req = 0000

code: PASS (expected 00, got 00)

valid: PASS (expected 0, got 0)

...

Test 10: req = 0101

code: PASS (expected 10, got 10)

valid: PASS (expected 1, got 1)

Test Summary:

Total tests run: 20

Number passed: 20

Number failed: 0

The notebook confirmed ['True'] for the fixed simulation, demonstrating that the generated testbench correctly distinguishes a buggy implementation from the correct one.

8. Simulation Summary

Run	Module	Type	Patterns	Comparisons	Passed	Failed	Status
Tutorial 1	mux2to1	Combinational	>20	>20	All	0	PASS
Tutorial 2	adder4bit	Arithmetic	10	20	20	0	PASS
Custom	priority_encoder4	Combinational	10	20	20	0	PASS
Bug Demo	prio_enc (buggy)	Combinational	10	20	14	6	FAIL
Bug Fixed	priority_encoder4	Combinational	10	20	20	0	PASS

Notes: 'Comparisons' counts individual port-level checks (one per output port per pattern). The adder has 2 outputs, so 10 patterns yield 20 comparisons. The priority encoder similarly has 2 outputs.

Grading Rubric

Criterion	Points	Status
Two tutorial examples pass final testbench	30	Complete
Custom module: end-to-end pipeline + passing simulation	30	Complete
Artifact understanding: golden model + patterns + updater	15	Complete
Bug detection: failing run + fix + passing run	15	Complete
Clarity & reproducibility: repo, notebook, report	10	Complete
TOTAL	100	

9. Discussion and Conclusions

9.1 Effectiveness of LLM-Driven Verification

Across all three module types, the pipeline produced syntactically valid Verilog testbenches and executable Python golden models without any manual editing. The golden models were particularly impressive: in all cases, the LLM correctly inferred the precise bit-width semantics (e.g., masking with 0xF for 4-bit operands, handling OR-reduction for the valid flag) purely from the natural-language description.

9.2 Prompt Engineering Observations

Three prompt choices had a significant impact on output quality:

- System vs. user role separation: Placing the role constraint in the system prompt and task description in the user prompt produced cleaner Verilog than combining both in a single message.
- Explicit port-name matching: Instructing the model to use the exact port names from the Verilog source as dict keys in the golden model was critical; without this, the model sometimes used aliases that broke the updater's key-based lookup.

- Markdown stripping: GPT-4o occasionally wraps code in ``` fences even when instructed not to. The post-processing strip ensures artifacts are always clean Verilog/Python, preventing silent compilation failures.

9.3 Test Coverage Analysis

For the MUX, the 8 possible input combinations are small enough that the LLM naturally produced exhaustive coverage. For the adder, 10 patterns out of 256 possible focused on corner cases (overflow boundary, max values) and representative mid-range values. For the priority encoder, all 10 patterns were distinct and covered all four single-bit cases plus multi-bit priority-resolution scenarios.

9.4 Bug Detection Power

The bug-detection experiment confirmed that the self-checking testbench is sensitive to even a single-bit output error in one case arm. The 6 failures precisely identified the three input patterns affected by the incorrect assignment, and the error message format ('expected 11, got 10') gave enough information to locate the bug without reading simulation waveforms.